

EM Inversion Workshop

GUI Manual

This manual describes how to set up a Python environment, configure machine-specific paths, install dependencies, and run the seven workshop GUIs. For each GUI it explains how to launch it, how to set parameters, and—for non-expert users—what each parameter means physically and what impact it has on the simulation or workflow.

The seven GUIs are:

- **00_configure_workshop** — Set rockem-suite path, MPI launcher, default SEG-Y, optional GPU engines for 2D forward/inversion, and other machine-specific settings.
- **01_fw_setup** — Prepare FD modelling inputs from a SEG-Y file.
- **02_fwmodelling_and_data_visualization** — Run FD modelling and visualize channel-gain amplitude/phase from shot records.
- **03_2d_inversion** — Generate inversion inputs and run the 2D inversion locally.
- **04_2d_inversion_results** — View 2D inversion results and compare real data with synthetics.
- **05_1d_inversion** — Independent 1D layered inversion, per transmitter, against a validated analytic magnetic (Hx) line-source solver.
- **06_1d_inversion_results** — View the 1D inversion’s pseudo-2D section, per-Tx diagnostics, and modelled-vs-observed data comparison.

Steps 00–04 form the 2D explicit-FDTD forward/inversion workflow; 05–06 are an independent, complementary 1D analysis that runs directly against the 2D forward run’s own data (no separate FDTD run needed) and does not depend on 03/04’s inversion having been run.

All GUIs are Jupyter notebooks served as code-hidden applications via Voila (ipywidgets and Plotly). Run Voila from the **workshop root** so that **scripts** and **third_party** are on the Python path.

1 Workspace layout

All notebook-generated artifacts are written under **workspace/** (gitignored). This keeps the repository root clean and separates 2D and 1D workflows:

```
workspace/  
  2D/  
    forward/          # Step 01-02: FD model, mod.cfg, shot gathers  
    inversion/  
      input/          # Step 03: prepared inversion inputs  
      Run{N}/         # Step 03: 2D inversion runs (Run0, Run1, ...)
```

```

    results/Run{N}/          # Step 04: SEG-Y exports
1D/
    inversion/Run{N}/        # Step 05: 1D inversion runs
    results/Run{N}/          # Step 06: SEG-Y exports

```

Example model: `examples/Fault_1.sgy` is a sample resistivity model committed with the repository. Load it in Step 01 (or set it as the default in Step 00).

Restore pristine state: run `./clean.sh` from the workshop root to remove `workspace/`, Voila PID files, and caches. Use `./clean.sh -dry-run` to preview, or `./clean.sh -y` to skip confirmation.

2 Environment setup

2.1 System prerequisites

Before setting up Python, ensure you have:

- Python 3.10 or newer
- `mpirun` (OpenMPI or compatible MPI runtime)
- A validated `rockem-suite` checkout, built with `make mpi`, with the **explicit** TE2D engine binaries `mpiEmmodTE2d` and `mpiEminvTE2d` under its `bin/` directory. Their location is `$ROCKEM_SUITE_ROOT/bin`, where `ROCKEM_SUITE_ROOT` is set by Step 00 (or defaults to `~/software/new_rockem/rockem-suite`; see `scripts/modules/rockem_bridge.py`). The ADI TE2D engine (`mpiEmmodADITE2d/mpiEminvADITE2d`) is **not** used by this workshop — it fails `rockem-suite`'s own layered-model Green's-function validation.

Optional — GPU acceleration (2D TE2D only): On a machine with an NVIDIA GPU and CUDA toolkit, you can build the GPU variants `mpiEmmodTE2dGpu` and `mpiEminvTE2dGpu` in `rockem-suite`:

```

make -f Makefile.gpu MPI_CXX=mpic++ RS_CUDA_ARCH=<arch> CUDA_HOME=$CUDA_HOME
\
bin/mpiEmmodTE2dGpu bin/mpiEminvTE2dGpu

```

Enable them in Step 00 (**Use GPU for 2D forward/inversion**). Step 00 **Validate** checks `nvidia-smi` and the GPU binaries when those options are on. GPU forward requires re-running Step 01 **Finalize setup** so `setup_metadata.json` records the GPU engine; GPU inversion is applied when Step 03 writes `runinv.sh`.

2.2 Creating a virtual environment (venv)

From the workshop root:

```

python3 -m venv .venv
source .venv/bin/activate    (Linux/macOS)
.venv\Scripts\activate      (Windows)

```

Then upgrade pip: `pip install -upgrade pip`.

2.3 Creating a conda environment

```

conda create -n em_workshop python=3.10
conda activate em_workshop

```

2.4 Installing dependencies

With the virtual or conda environment activated, install:

Package	Purpose
voila	Serves notebooks as standalone web apps (code hidden).
ipywidgets	Interactive widgets in the GUI.
plotly	Interactive plots.
numpy	Numerical arrays (used by inversion and results GUIs).
ipykernel	Jupyter kernel for Python (required for Voila to run notebooks).
matplotlib	Plotting (used by workshop modules).
scipy	Scientific computing (used by workshop modules).
segvio	Read/write SEG-Y files (required by 01_fw_setup).
joblib	Parallelizes 05's $T_x \times$ seed ensemble inversion across CPU cores.

Command:

```
pip install voila ipywidgets plotly numpy ipykernel matplotlib scipy
segvio joblib
```

Conda users: After installing, register the kernel so Voila can find it:

```
python -m ipykernel install -user -name=em_workshop -display-name="Python
(em_workshop)"
```

3 How to run the GUIs

From the **workshop root** directory, use the provided shell scripts. Each script changes into the script directory and starts Voila with the corresponding notebook (sources stripped so users see only the interface).

- `./start_00_configure.sh` → `00_configure_workshop`
- `./start_01_fw_setup.sh` → `01_fw_setup`
- `./start_02_fwmodelling_and_data_visualization.sh` → `02_fwmodelling_and_data_visualization`
- `./start_03_2d_inversion.sh` → `03_2d_inversion`
- `./start_04_2d_inversion_results.sh` → `04_2d_inversion_results`
- `./start_05_1d_inversion.sh` → `05_1d_inversion`
- `./start_06_1d_inversion_results.sh` → `06_1d_inversion_results`

Recommended order: run Step 00 once per machine, then `01` → `02` → `03` → `04` for the 2D workflow. Steps 05–06 can follow after 01/02 have produced FD data for at least one transmitter — they read directly from that data and do not require 03/04's 2D inversion to have been run.

Alternatively, from the workshop root:

```
voila 00_configure_workshop.ipynb -strip_sources=True
```

(and similarly for the other notebooks.)

Each start script writes its own Voila PID file (e.g. `.voila_fw_setup_server.pid`, `.voila_2d_inversion_server.pid`), so multiple GUIs can be managed independently.

4 Recommended workflow

1. **Step 00 — Configure** — Set `rockem-suite` root, MPI launcher (`mpirun`), default process count, default SEG-Y (`examples/Fault_1.sgy`), workspace directory name, and optional GPU flags for 2D forward (`mpiEmmodTE2dGpu`) and inversion (`mpiEminvTE2dGpu`). Click **Validate** to check CPU/GPU binaries, `nvidia-smi`, and paths, then **Save** to write `workshop_config.json`.
2. **Step 01 — FW setup** — Load a SEG-Y resistivity model, configure survey and FD design, then **Generate FD inputs (Finalize setup)**. Outputs go to `workspace/2D/forward/`.
3. **Step 02 — FW modelling and data visualization** — Run forward modelling, load `Hxshot.rss/Hzshot.rss`, compute channel-gain amplitude/phase, and inspect results.
4. **Step 03 — 2D inversion** — Generate inversion inputs into `workspace/2D/inversion/input/`, then run inversion into `workspace/2D/inversion/RunN/`.
5. **Step 04 — 2D inversion results** — Compare models and data; export SEG-Y to `workspace/2D/results/RunN/`.
6. **Step 05 — 1D inversion** — Run per-transmitter 1D layered inversion; outputs go to `workspace/1D/inversion/RunN/`.
7. **Step 06 — 1D inversion results** — View pseudo-2D sections and export to `workspace/1D/results/RunN/`.

5 GUI 0: 00_configure_workshop

5.1 Purpose

Configure machine-specific paths before running the other notebooks. Settings are saved to `workshop_config.json` at the repo root (gitignored). All other notebooks read this file at startup via `scripts/modules/workshop_config.py`.

5.2 Parameters

- **rockem-suite** — Path to the validated rockem-suite checkout (must contain `python/`, `bin/mpiEmmodTE2d`, `bin/mpiEminvTE2d`, and the analytic Green's-function solver under `doc/examples/validate_layered_1d_model/shared/`).
- **mpirun** — MPI launcher executable (usually `mpirun`). Used by Steps 02, 03, and 04 when launching FD or inversion jobs.
- **nproc** — Default number of MPI processes for FD and inversion runs (can still be changed per GUI).
- **default SEG-Y** — Default resistivity model path pre-filled in Step 01 and used as fallback for SEG-Y export in Steps 04 and 06. The workshop includes `examples/Fault_1.sgy`.
- **workspace** — Name of the generated-artifacts directory (default `workspace`).
- **Use GPU for 2D forward** — When enabled, Steps 01, 02, and 04 use `mpiEmmodTE2dGpu` instead of `mpiEmmodTE2d`. Requires the GPU binary and a working NVIDIA driver (`nvidia-smi`). After enabling, re-run Step 01 **Finalize setup**.
- **Use GPU for 2D inversion** — When enabled, Step 03 writes `runinv.sh` with `mpiEminvTE2dGpu` instead of `mpiEminvTE2d`.

- **Buttons** — **Validate** checks CPU binaries (required), GPU binaries and `nvidia-smi` (required when GPU options are on; informational otherwise), `mpirun`, default SEG-Y, and the analytic solver; **Save** writes `workshop_config.json`; **Reset defaults** restores built-in defaults without saving; **Quit Voila** stops the server.

5.3 Configuration file fields

When saved, `workshop_config.json` includes:

- `rockem_suite_root`, `mpirun`, `nproc_default`, `default_seg_y`, `workspace_dir`
- `use_gpu_forward_2d` — `true/false`
- `use_gpu_inversion_2d` — `true/false`

6 GUI 1: 01_fw_setup

6.1 Purpose

Generate finite-difference (FD) modelling inputs from a SEG-Y resistivity model: conductivity/permittivity files (`sg.rss`, `ep.rss`), source wavelet (`wav2d.rss`), survey geometry (`Survey.rss`), and `mod.cfg` settings. Intermediate files are written to a temporary session folder; final outputs go to `workspace/2D/forward/`.

6.2 Parameters (physical meaning and impact)

- **SEG-Y file** — Path to the input SEG-Y file containing the resistivity model. Defaults to `examples/Fault_1.sgy` if configured in Step 00. Uniform permittivity for `ep.rss` is set from the FD design (`eps_r_used` in section 4 / `setup_metadata.json`), not from a separate constant.
- **tx0 (m), tz0 (m), dtx (m), ntx** — Transmitter (source) geometry: first source position in x and z , spacing between sources, and number of sources. All positions must lie inside the model bounds.
- **rx0 (m), rz0 (m), drx (m), nrx** — Receiver geometry: first receiver x and z , receiver spacing, and number of receivers.
- **flist (Hz)** — Comma-separated list of source frequencies (e.g. `2000,4000,6000`). Lower frequencies penetrate deeper; higher frequencies resolve shallow structure better.
- **dt (s)** — Time step for the wavelet and for the FD solver. It must satisfy numerical stability (CFL) and sample the highest frequency.
- **n_periods** — Number of periods of the lowest frequency in the wavelet. Controls how long the source radiates.
- **alpha** — Ramp parameter (0–1) for the ramp square-wave wavelet.
- **f_min (Hz), f_max (Hz)** — Frequency range used to design the FD grid for the **explicit** engine (`mpiEmmodTE2d` or `mpiEmmodTE2dGpu` when GPU forward is enabled in Step 00). Grid spacing, time step, `eps_r`, PML, and depth margin follow deterministically from these plus `rho_min/rho_max` and receiver offsets.
- **rho_min (Ohm-m), rho_max (Ohm-m)** — Expected resistivity range in the model. Grid spacing is sized from skin depth and minimum receiver offset; `eps_r` is sized from `rho_max/f_max`.

- **dimension** — 2D or 3D. **Note:** the 3D path uses the legacy ADI engine and is not covered by this workshop’s explicit-engine validation.
- **points/skin, cells/min offset, tan(delta) floor, eps_r cap, sigma_max clip** — FD design controls; see the GUI readout after **Compute FD design**.
- **Depth margin (auto)** — If the loaded model is too shallow for source/receiver clearance, **Generate FD inputs** automatically pads it in depth.
- **Buttons** — **Load model**, **Plot model**, **Build intermediate sg/ep**, **Build intermediate wav2d**, **Generate survey.cfg + Survey.rss**, **Compute FD design**, **Generate FD inputs (Finalize setup)** writes everything to `workspace/2D/forward/`. Use **Quit GUI server** to stop Voila.

7 GUI 2: 02_fwmodelling_and_data_visualization

7.1 Purpose

Run FD modelling (via `workspace/2D/forward/runmod.sh` and the explicit-engine MPI binary selected in Step 00), load `Hxshot.rss` and `Hzshot.rss` outputs from `workspace/2D/forward/Data/`, compute steady-state channel-gain amplitude and phase, and visualize or save the results.

7.2 External requirement

The explicit-engine MPI FD binary must be installed: `mpiEmmodTE2d` (CPU) or `mpiEmmodTE2dGpu` (GPU). The engine name is stored in `workspace/2D/forward/setup_metadata.json` when Step 01 finalizes setup. Step 00 (or `ROCKEM_SUITE_ROOT`) must point at a valid checkout; `runmod.sh` reads the engine from that metadata.

7.3 Parameters (physical meaning and impact)

- **nproc** — Number of MPI processes for the FD run (default from Step 00 config).
- **f_min (Hz), n_periods (of f_min)** — Reference frequency and extraction window for steady-state phasor extraction (same convention as Step 05).
- **component** — Hx or Hz magnetic field component.
- **plot (metric)** — Amplitude or phase vs receiver, transmitter, or frequency.
- **frequency, tx, local rx** — Selectors for display.
- **Buttons** — **Run modelling (background)**, **Stop run**, **Refresh run status**, **Load FD outputs**, **Compute channel-gain amplitude/phase**, **Save processed results** (writes NPZ under `Data/processed/`), **Quit GUI server**.

8 GUI 3: 03_2d_inversion

8.1 Purpose

Generate inversion inputs from `workspace/2D/forward/` and the inversion template, run the inversion locally (staged into the next `workspace/2D/inversion/RunN/` directory), and monitor logs and model progress.

8.2 External requirement

The explicit-engine MPI inversion binary must be installed: `mpiEminvTE2d` (CPU) or `mpiEminvTE2dGpu` (GPU). Step 03 writes `runinv.sh` with the engine selected in Step 00 (`scripts/modules/workshop_config.patch_runinv_template`).

8.3 Parameters (physical meaning and impact)

- **Initial model** — Uniform conductivity (sigma) or resistivity (rho) starting model.
- **sigma (S/m), rho (Ohm.m)** — Value of the uniform initial model.
- **Max iter** — Maximum inversion iterations.
- **apertx (m), dtx, dtz** — Grid and extent parameters. **apertx** defaults to the forward run's value from Step 01 when available.
- **Tikhonov (sg)** — Regularization weight (`tik_sgregalpha`).
- **Clean selected RunN before run** — Clear the chosen run directory before staging new inputs.
- **Load run** — Select which RunN under `workspace/2D/inversion/` to monitor.
- **nproc** — MPI processes for inversion (default from Step 00).
- **Buttons** — **Generate inversion inputs** writes to `workspace/2D/inversion/input/`; **Run inversion locally** stages into `workspace/2D/inversion/RunN/`; **Refresh from progress.log**; **Stop run**; **Quit GUI server**.

9 GUI 4: 04_2d_inversion_results

9.1 Purpose

View 2D inversion results: select a run (RunN) and model (`sg0`, `sg_ls` at an iteration, etc.), view 2D resistivity with true-model comparison, resistivity slices, compare real data with synthetics, and export selected models to SEG-Y.

9.2 Parameters (physical meaning and impact)

- **Run, Model** — Which inversion run under `workspace/2D/inversion/` and which model file.
- **Export selected model to SEG-Y** — Writes to `workspace/2D/results/RunN/` using the setup SEG-Y as a template. Interpolates to template geometry if needed.
- **Export naming** — e.g. `sg_up_iter011.segy`, `sg0_iter000.segy`, `sg_ls_iterNNN.segy`.
- **x (m), z (m), Freqs (Hz)** — Slice positions and frequencies for comparison.
- **Load real data / Load synthetics from run / Generate synthetics from model** — Data comparison options. **Generate synthetics** runs FD for the selected model using `nproc` MPI processes. It always refreshes `ep.rss` from `workspace/2D/forward/` and checks that its permittivity matches `eps_r_used` in `setup_metadata.json` from Step 01 (not the legacy ADI default of 7).
- **component, plot, frequency, tx, local rx, trace idx** — Display selectors (same as Step 02).
- **Quit GUI server** — Stops Voila.

10 GUI 5: 05_1d_inversion

10.1 Purpose

An independent 1D layered inversion, run separately for each transmitter, against rockem-suite's validated analytic magnetic (Hx) line-source Green's function. Reads channel-gain data from `workspace/2D/forward/Data/` (same phasor extraction as Step 02) — it does not need Steps 03/04 to have been run. Each transmitter is calibrated against the analytic solver on the blocky 1D model beneath it, then inverted independently; results are combined into a pseudo-2D section under `workspace/1D/inversion/RunN/`.

10.2 Parameters (physical meaning and impact)

- **Freqs (Hz), n_periods (of f_min)** — Channel-gain extraction settings (same convention as Step 02).
- **n_layers, z_start, z_end, rho_bg, rho_min, rho_max, thk_min, thk_max** — 1D model parameterization and search bounds.
- **optimizer** — Differential Evolution, Dual Annealing, or BlockInv (Gauss-Newton).
- **calibration** — Fixed calibration constant (recommended) or per-frequency source factor.
- **maxiter, popsize, maxfun, seed, N runs/Tx, n_jobs, blk_iter** — Optimizer and ensemble settings.
- **w_hxh, w_hxhz, Tikhonov lambda** — Misfit weights and smoothness penalty.
- **dx_out, dz_out, zmax (0=auto)** — Pseudo-2D output grid.
- **Buttons** — Reload features, Check kx convergence, Run all Tx inversions, Build pseudo-2D section, Export results (NPZ/JSON under current RunN), Refresh QC plots, Quit GUI server.

11 GUI 6: 06_1d_inversion_results

11.1 Purpose

Visualize and export 1D inversion results from Step 05: pseudo-2D section (mean and uncertainty), per-transmitter diagnostics, and modelled-vs-observed data comparison.

11.2 Parameters (physical meaning and impact)

- **Run** — Which RunN under `workspace/1D/inversion/` to load.
- **x slice (m), z slice (m), bg rho (Ohm.m)** — Section display controls.
- **Tx, Uncertainty** — Per-transmitter diagnostics and uncertainty band (1 std or 90% CI).
- **Freqs (Hz), component, plot, frequency, local rx, trace idx** — Data-comparison panel (same as Step 04).
- **Buttons** — Load run, Refresh run list, Rebuild section from saved layers, Refresh Tx plots, Load real data, Generate synthetics from inverted model, Export mean section to SEG Y (writes to `workspace/1D/results/RunN/`), Quit GUI server.

12 Troubleshooting

- **Missing configuration** — Run Step 00 and click **Save** so `workshop_config.json` exists. Without it, defaults apply (`~/software/new_rockem/rockem-suite, examples/Fault_1.sgy`).
- **Missing segyio** — Install `segyio` and start Voila from the workshop root.
- **No Jupyter kernel for language ‘python’ found** — Install `ipykernel` and register the `em_workshop` kernel (see Environment setup).
- **SEG-Y export cannot find template** — Export expects `segy_template_path` in `workspace/2D/forward/setup_metadata.json`, with fallback to the default SEG-Y from Step 00. Run Step 01 and finalize setup first.
- **Missing explicit-engine MPI FD binary** — Steps 02 and 04 need `mpiEmmodTE2d` (or `mpiEmmodTE2dGpu` if GPU forward is enabled). Validate in Step 00; build CPU with `make mpi`, GPU with `make -f Makefile.gpu bin/mpiEmmodTE2dGpu`.
- **Missing explicit-engine MPI inversion binary** — Step 03 needs `mpiEminvTE2d` (or `mpiEminvTE2dGpu` if GPU inversion is enabled).
- **GPU validation fails** — Ensure `nvidia-smi` works, CUDA is installed, and GPU binaries exist under `$ROCKEM_SUITE_ROOT/bin/`. Build targets in `rockem-suite` as shown in Environment setup. If you enabled GPU forward after Step 01, re-run **Finalize setup** in Step 01.
- **Wrong/stale rockem-suite checkout** — Step 00 validation checks for `magnetic_line_source_fields_` in the analytic solver shared directory.
- **Missing joblib** — Step 05 needs `joblib` for parallel ensemble runs.
- **Stale artifacts** — Run `./clean.sh` to remove `workspace/` and start fresh.

13 File locations

- `workshop_config.json` — Machine-specific settings (Step 00; gitignored): `rockem-suite` path, MPI launcher, default SEG-Y, workspace dir, `use_gpu_forward_2d`, `use_gpu_inversion_2d`.
- `workspace/2D/forward/` — FD inputs and outputs (`mod.cfg`, `sg.rss`, `Survey.rss`, `Data/`).
- `workspace/2D/inversion/input/` — 2D inversion input files (`inv.cfg`, `sg0.rss`, etc.).
- `workspace/2D/inversion/Run0/`, `Run1/`, ... — 2D inversion run directories (Steps 03/04).
- `workspace/2D/results/RunN/` — SEG-Y exports from Step 04.
- `workspace/1D/inversion/Run0/`, ... — 1D inversion run directories (Steps 05/06).
- `workspace/1D/results/RunN/` — SEG-Y exports from Step 06.
- `examples/Fault_1.sgy` — Sample resistivity model (committed).
- `scripts/modules/` — Python helpers (`workshop_config.py`, `rockem_bridge.py`, SEG-Y I/O, FD design, 1D analytic forward, 2D inversion helpers); see `scripts/README.md`.
- `scripts/templates/` — Templates for `inv.cfg`, `mod.cfg`, `runmod.sh`, `runinv.sh`, etc.
- `scripts/clean_workspace.py`, `clean.sh` — Remove all generated artifacts.

- `third_party/` — Vendored Rockseis Python code and BlockInv 1D inversion helpers.